E

ELKATECH LAUNCHPAD

Database Schema Design

ElkaTech Launchpad — Service Platform

Document: Database Schema Design
Project: ElkaTech Launchpad
Version: v2.0
Date: 11 June 2026
Status: Current — feature/owner-support-roles

Table of Contents

1. Schema Overview

ElkaTech Launchpad uses a single PostgreSQL database divided into four service schemas plus a migration-tracking table. Everything below is derived directly from the SQL migrations under `services//migrations/.sql`; nothing is invented.

SCHEMA	OWNER SERVICE	TABLES
`auth`	auth	`users`, `sessions`, `tokens`, `oauth_identities`, `outbox`
`catalog`	catalog	`categories`, `products`
`service_desk`	service-desk	`requests`, `request_messages`, `request_history`, `request_attachments`, `customer_machines`, `outbox`
`notification`	notification	`deliveries`
`public`	tooling	`platform_migrations` (applied-migration ledger)

Cross-schema references (a request's `customer_id`, `assigned_engineer_id`, and `customer_machine_id`) are stored as bare UUIDs **without** foreign keys, by design — ownership is enforced in the application layer to respect service boundaries.

2. Auth Schema

auth.users

ASPECT	DETAIL
Purpose	Portal accounts for all roles
Key fields	`id` (uuid PK), `email` (unique), `display_name`, `role`, `password_hash` (nullable), `email_verified`, `status`
Identity / approval	`firebase_uid` (unique where present), `approval_status`, `approved_at/by`, `rejected_at/by`, `suspended_at/by`, `account_origin`
Soft delete	`removed_at`, `removed_by`

ASPECT	DETAIL
Profile (mig. 006)	`company_name`, `contact_phone`, `alternate_phone`, `address_line1/2`, `city`, `state`, `postal_code`, `country`, `profile_completed`, `profile_completed_at`
Notes	Service profile lives on the user row so it flows through session resolve with no extra join

auth.sessions

ASPECT	DETAIL
Purpose	Active login sessions
Key fields	`id`, `token_hash` (unique), `csrf_token`, `user_id` → `auth.users` (cascade), `user_agent`, `ip_address`, `expires_at`, `last_seen_at`

auth.tokens

ASPECT	DETAIL
Purpose	One-time tokens for verify/reset/invite
Key fields	`id`, `token_hash` (unique), `user_id` → `auth.users` (cascade, nullable), `email`, `role`, `purpose`, `expires_at`, `consumed_at`
Constraints	`purpose ∈ (verify_email, reset_password, invite_signup)`

auth.oauth_identities (migration 002)

ASPECT	DETAIL
Purpose	Link external providers (Google) to a local user
Key fields	`id`, `user_id` → `auth.users` (cascade), `provider`, `provider_user_id`, `provider_email`, `unique(provider, provider_user_id)`

auth.outbox

ASPECT	DETAIL
Purpose	Auth domain events awaiting notification
Key fields	`id`, `aggregate_type`, `aggregate_id`, `event_type`, `payload` (jsonb), `occurred_at`, `processed_at`

3. Catalog Schema

catalog.categories

ASPECT	DETAIL
Purpose	Product categories
Key fields	`id` (text PK), `slug` (unique), `route_path` (unique), `name`, `intro`

catalog.products

ASPECT	DETAIL
Purpose	Catalog products (general models for sale)
Key fields	`id` (text PK), `category_id` → `catalog.categories` (cascade), `category_slug`, `slug` (unique), `name`, `price_display`, `brochure_url`, `images` / `specs` / `highlights` (jsonb)

4. Service Desk Schema

service_desk.requests

ASPECT	DETAIL
Purpose	Service requests and their lifecycle
Key fields	`id`, `request_number` (unique), `customer_id`, `customer_email`, `product_id`, `product_snapshot` (jsonb), `subject`, `description`, `contact_phone`, `site_location`, `serial_number`, `priority`, `status`, `assigned_engineer_id`
Machine link (mig. 003)	`customer_machine_id` (nullable), `issue_type`
Relationships	`customer_id` / `assigned_engineer_id` → `auth.users` (app-enforced); `customer_machine_id` → `customer_machines` (app-enforced)

service_desk.customer_machines (migration 002)

ASPECT	DETAIL
Purpose	One physical owned unit per row (admin/owner controlled)
Key fields	`id`, `customer_id`, `product_id`, `product_snapshot` (jsonb), `display_label`, `unit_number`, `internal_serial_number` (staff-only), `site_name`, `site_location`, `contact_phone`, `purchase_date`, `install_date`, `status`, `notes`
Notes	Status `active` / `inactive`; customer-facing view hides internal serial and notes

service_desk.request_messages

ASPECT	DETAIL
Purpose	Conversation: customer-visible messages and internal notes
Key fields	`id`, `request_id` → `requests` (cascade), `author_id`, `author_role`, `visibility`, `body`
Constraints	`visibility ∈ (customer_visible, internal_note)`; `author_role ∈ (customer, engineer, admin)` — see Section 10 note

service_desk.request_history

ASPECT	DETAIL
Purpose	Activity timeline of key request events
Key fields	`id`, `request_id` → `requests` (cascade), `actor_id`, `actor_role`, `event_type`, `metadata` (jsonb)
Constraints	`actor_role ∈ (customer, engineer, admin)` — see Section 10 note

service_desk.request_attachments (migration 004)

ASPECT	DETAIL
Purpose	Metadata for photo/video evidence stored in Cloudflare R2
Key fields	`id`, `request_id` → `requests` (cascade), `uploaded_by`, `object_key` (unique), `file_name`, `content_type`, `size_bytes`, `kind`
Constraints	`kind ∈ (image, video)`; bytes live in R2, only metadata in Postgres

service_desk.outbox

ASPECT	DETAIL
Purpose	Service-desk domain events awaiting notification
Key fields	`id`, `aggregate_type`, `aggregate_id`, `event_type`, `payload` (jsonb), `occurred_at`, `processed_at`

5. Notification Schema

notification.deliveries

ASPECT	DETAIL
Purpose	Record of emails sent per outbox event
Key fields	`id`, `event_id`, `event_type`, `recipient_email`, `subject`, `status`, `attempts`, `last_error`, `sent_at`
Constraints	`status ∈ (sent, failed)`; unique `(event_id, recipient_email)` (migration 002) for per-recipient idempotency

6. Role, Status, and Enum Fields

FIELD / TABLE	ALLOWED VALUES
`auth.users.role`, `auth.tokens.role`	`customer`, `engineer`, `support`, `owner`, `admin` (relaxed by migration 008)
`auth.users.status`	`active`, `invited`
`auth.users.approval_status`	`pending_approval`, `approved`, `rejected`, `suspended`
`auth.users.account_origin`	`self_signup`, `admin_invite`, `firebase_google`, `legacy`
`auth.tokens.purpose`	`verify_email`, `reset_password`, `invite_signup`
`requests.priority`	`low`, `normal`, `high`, `urgent`
`requests.status`	`new`, `triaged`, `assigned`, `in_progress`, `waiting_for_customer`, `resolved`, `closed`
`request_messages.visibility`	`customer_visible`, `internal_note`
`customer_machines.status`	`active`, `inactive`
`request_attachments.kind`	`image`, `video`
`deliveries.status`	`sent`, `failed`
`requests.issue_type`	Free-text column; validated in the app against the issue-type contract (e.g. `printing_issue`, `ink_issue`, `other`)

7. Important Indexes

TABLE	INDEX	PURPOSE
`auth.users`	unique `email`; partial unique `firebase_uid`; partial `removed_at` (where null)	Lookup and active-account filtering
`auth.sessions` / `auth.tokens`	unique `token_hash`	Constant-time session/token resolve
`auth.oauth_identities`	unique `(provider, provider_user_id)`	One identity per provider account
`catalog.products`	unique `slug`; FK on `category_id`	Routing and category joins
`customer_machines`	`customer`, `product`, `status`, `(customer, status)`	Dashboard and active-machine filters

TABLE	INDEX	PURPOSE
`requests`	unique `request_number`; `customer_machine_id`	Reference and machine-scoped lookups
`request_attachments`	unique `object_key`; `request_id`	R2 key uniqueness and per-request listing
`notification.deliveries`	unique `(event_id, recipient_email)`	Retry idempotency

8. Relationships (ERD Summary)

```
auth.users 1──* auth.sessions          (FK, cascade)
auth.users 1──* auth.tokens            (FK, cascade)
auth.users 1──* auth.oauth_identities  (FK, cascade)
catalog.categories 1──* catalog.products (FK, cascade)
service_desk.requests 1──* request_messages    (FK, cascade)
service_desk.requests 1──* request_history     (FK, cascade)
service_desk.requests 1──* request_attachments (FK, cascade)
service_desk.customer_machines *──1 auth.users  (app-enforced uuid, no FK)
service_desk.requests          *──1 auth.users  (customer + engineer, app-enforced)
service_desk.requests          *──1 customer_machines (app-enforced)
auth.outbox / service_desk.outbox ──poll──> notification.deliveries
```

9. Data Lifecycle

- **Outbox → delivery:** events are written in the same transaction as the domain change; the notification poller marks `processed_at` and records a delivery per recipient.
- **Requests:** progress through the status enum; `closed` represents archived work. History rows accumulate per event.
- **Migrations:** every applied file is recorded in `public.platform_migrations` so `db:migrate` is idempotent.

10. Soft Delete vs Hard Delete

- **Users — soft:** `removed_at` / `removed_by` mark a user as removed for list filtering without losing the row immediately.
- **Users — hard (admin only):** the permanent-delete path removes the user and cascades sessions, tokens, OAuth identities, and the requests they created (with each request's messages/history/attachments), in a single transaction.
- **Machines — soft:** status is set to `inactive`; rows and request history are preserved and can be reactivated.

> Note: `request_messages.author_role` and `request_history.actor_role` still enumerate only `customer`, `engineer`, and `admin` — these checks predate the owner/support roles (migration 008 relaxed only `auth.users.role` and `auth.tokens.role`). This is documented as the current schema state, not a recommendation.

11. R2 Object Key / Attachment Mapping

- `request_attachments.object_key` is the unique Cloudflare R2 object key; the binary bytes live in R2, not Postgres.
- Object keys are request-prefixed and validated on confirmation so an upload cannot be attached to a different request.
- Read URLs are derived at access time from `R2_PUBLIC_BASE_URL` (when set) or a short-lived presigned `GET`; the URL is never stored, so rotating the bucket or base URL requires no data migration.